

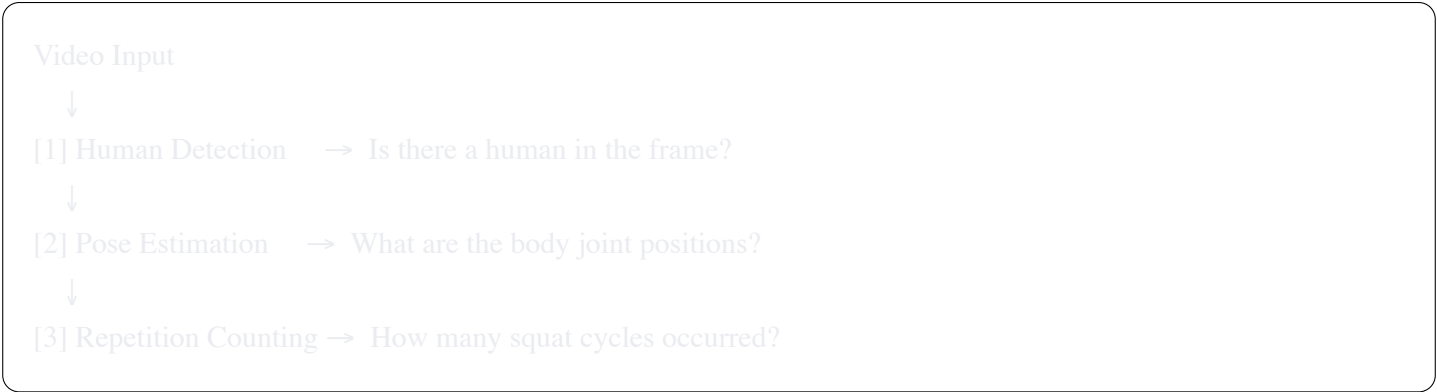
RepCount: Squat Rep Counter — Full DS Project Lifecycle

Phases 1–4 Deep Dive | Prototype to Mobile

◆ PHASE 1 — Problem Definition & Scoping

What Are We Actually Building?

This is **NOT a classification task**. It is a compound task of three sequential sub-tasks:



- **Classification per video or per frame?** Neither — it is **temporal localization + regression**. The model must detect *when* each rep starts and ends (frame-level resolution), and *count* how many occurred. This is fundamentally a temporal sequence problem.
- **Prototype scope:** Squats only, single-person videos, offline video input (not real-time yet).
- **Mobile scope (Phase 2+):** Real-time camera feed → pose → count displayed live.

Architecture Artifacts (Phase 1)

Artifact	Description	Owner
Problem Statement Doc	1-pager: task definition, scope, out-of-scope	DS Lead
Feasibility Assessment	Can humans do this task reliably? IAA check	DS Lead
Metrics Framework	Training metrics + deployment KPIs defined	DS + PM
Annotation Audit Report	Verify existing labels are correct (see Phase 4)	DS

Architecture Decision Record (ADR)	TransRAC vs PoseRAC vs custom pipeline	DS Lead
Mobile Constraints Doc	Latency budget, device specs, framework choice	DS + Mobile Eng

Meetings Required (Phase 1)

Meeting	Attendees	Output
Kickoff	DS, PM, Eng, Mobile	Aligned on scope and success criteria
Annotation Audit Review	DS + Domain Expert	Confirm annotation quality
Architecture Spike Review	DS team	Decide on model approach for prototype
Feasibility Gate 🚦	DS Lead + Stakeholders	Go/No-Go on prototype → if green, add 3 resources

Metrics: Three Levels

Level 1 — Model Training Metrics (Is the model learning?)

Metric	Formula	Target
MAE (Mean Absolute Error)	$\frac{1}{n} \sum y_i - \hat{y}_i $	predicted_count - true_count
OBO (Off-By-One Accuracy)	$\frac{1}{n} \sum \mathbb{1}_{ y_i - \hat{y}_i = 1}$	pred - true
MAPE	$\frac{1}{n} \sum \frac{ y_i - \hat{y}_i }{ y_i }$	pred-true
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Track for regression stability

OBO is the PRIMARY metric for RepCount. The benchmark paper (TransRAC) reports OBO = 0.29, and PoseRAC achieves OBO = 0.56. Your prototype target should be OBO > 0.50.

Level 2 — Deployment/Performance Metrics (Is the deployed model performing?)

Metric	Description	Target
Inference latency	ms per frame on target device	< 100ms/frame mobile
FPS throughput	Frames processed per second	≥ 15 FPS mobile
Model size	MB on disk after quantization	< 20MB (mobile)
Error rate in production	% videos with count error > 3	< 10%
Drift detection	MAE tracked on rolling 7-day window	Alert if MAE rises > 20%

Level 3 — Human Evaluation (Is the product experience correct?)

Test	Method	Target
Expert review	Fitness trainers count reps manually vs model	Agreement > 90%
User acceptance	Real users with app: "Did the counter feel correct?"	> 4/5 rating
Edge case audit	Partial reps, pauses mid-set, multiple people	Document failure modes
Speed sensitivity	Fast vs slow reps: does the model still count correctly?	OBO > 0.45 on fast reps

KPIs (Business / Product Level)

A KPI connects model performance to a business or product outcome. They are *not* the same as model metrics.

KPI	Definition	Example Target
Counting Accuracy Rate	% of sets where count is exact	> 75% at launch
User Retention (7-day)	% users who use the app again after Day 1	> 40%
Session rep-count volume	Total reps counted per DAU per day	Baseline → track growth
Time-to-first-count	Seconds from opening camera to first rep detected	< 3 seconds

Crash/Fail rate	% sessions where model failed to initialize or crashed	< 1%
Generalization coverage	% of supported exercises with OBO > 0.50	Start with 1 (squat), scale to 5

Example KPI narrative: "By Q2, 75% of squat sessions tracked by the app will have a rep count within ± 1 of the user's manual count, as verified by a monthly sample audit of 200 sessions."

Feasibility Check: Can We Do This?

Answer: YES — Green. Here's why:

- ✓ RepCount provides 1,041 labeled videos for Part-A (squats included)
- ✓ Pose-based approaches (PoseRAC, MediaPipe + custom) are proven on this exact dataset
- ✓ MediaPipe Pose runs at real-time on mobile (30 FPS on iPhone 12+)
- ✓ PoseRAC trains in 20 min on a single GPU — prototype is fast to iterate
- ⚠ Class imbalance (squats dominant — addressed in Phase 2)
- ⚠ Single-person assumption must hold (multi-person is a known failure mode per TransRAC paper)

Prototype stack decision: MediaPipe Pose → joint angle features → lightweight LSTM/Transformer counter

This is faster to iterate than end-to-end video models, mobile-friendly, and interpretable.

◆ PHASE 2 — Data Sourcing & EDA (Squat Focus)

Class Imbalance

The RepCount dataset is heavily skewed. Squats are the most represented class — this is actually **advantageous for your prototype** but means:

- Don't evaluate overall accuracy on the full dataset (it will be inflated by squat dominance)
- Always report **per-class metrics** when you scale to other exercises
- Use **stratified sampling** when creating train/val splits

EDA Deliverables

- Distribution of `count` column (histogram of reps per video — squats only)
- Distribution of clip duration (seconds) for squat videos
- Frame rate consistency check (is it 30fps? variable fps?)
- Annotation density: how many start/end pairs per video on average?
- Outlier detection: videos with count > 30 or < 2 (anomaly cases per dataset design)

Meetings

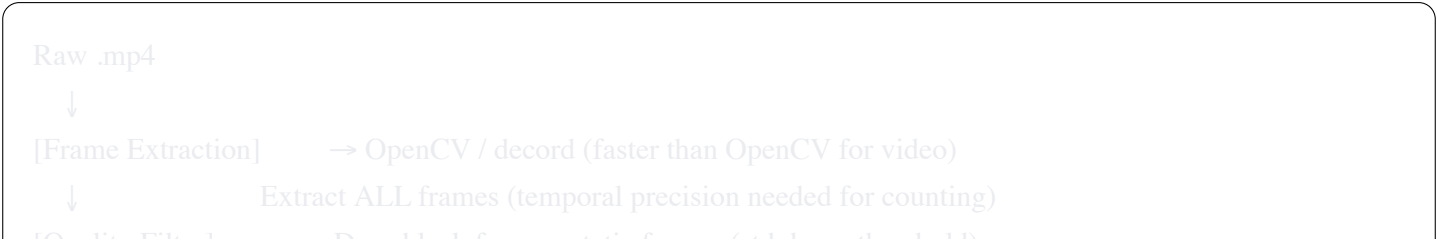
Meeting	Output
EDA Readout	Team aligned on data quality, imbalance strategy, outlier treatment
Dataset Augmentation Decision	Do we add OVR dataset (72K videos)? Decision: yes for Phase 2, prototype on RepCount first

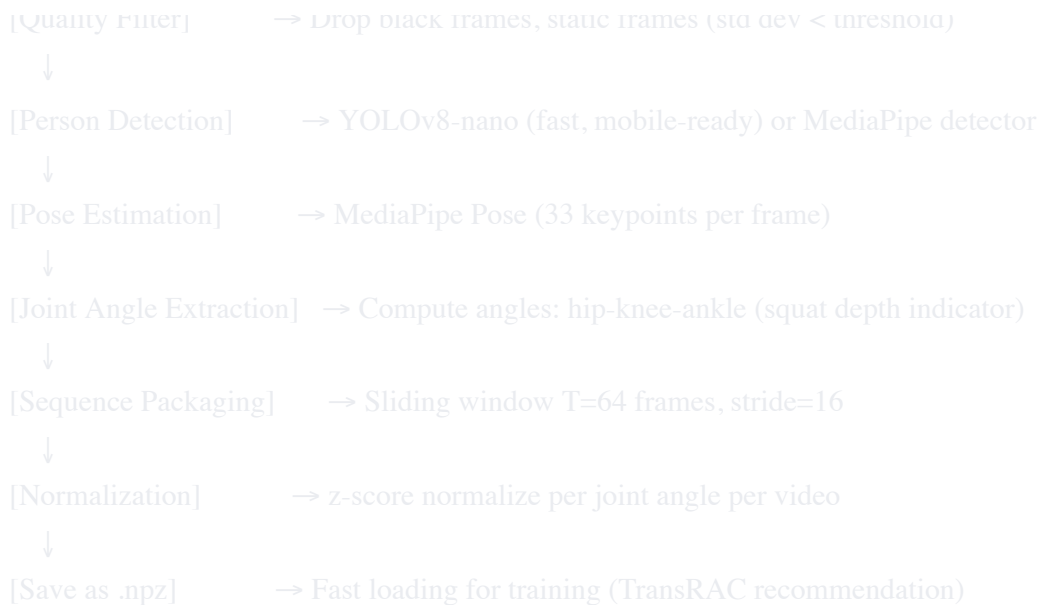
◆ PHASE 3 — Preprocessing Pipeline (mp4, No Audio)

Confirmed from Dataset

-  **Format:** `.mp4` (confirmed by TransRAC GitHub — also supports `.npz` for preprocessed speed)
-  **Audio:** Not used — no audio track in our pipeline
- **Average length:** ~39 seconds per video
- **Recommended frame extraction:** All frames or at fixed rate (e.g., 30fps native)

Pipeline





Key Preprocessing Decisions

- **Do NOT downsample aggressively** — for counting, temporal resolution matters. Use native FPS.
- **Single-person crop**: If multiple people detected, take the largest bounding box (dominant subject heuristic).
- **No audio pipeline needed** — skip completely.

Artifacts

- Preprocessing script (Python, reproducible)
- Preprocessing config YAML (fps, window size, normalization params)
- Data quality report (% frames dropped, % videos with no person detected)

◆ PHASE 4 — Annotation Analysis & Validation

What Kind of Annotations Do We Have?

Answer: Frame-level temporal annotations. NOT bounding boxes.

Based on confirmed research (TransRAC CVPR 2022 + dataset page):

Each row in train.csv / valid.csv / test.csv:

```
video_id | location | num_frames | count | start_1 | end_1 | start_2 | end_2 |
```

video_id | full_action | num_frames | count | start_1 | end_1 | start_2 | end_2 | ...

start_X / end_X = FRAME NUMBER (integer)

NOT seconds, NOT bounding box coordinates

So for a squat video with 3 reps and 900 frames:

```
name | count | start_1 | end_1 | start_2 | end_2 | start_3 | end_3
squat_01 | 3 | 45 | 135 | 180 | 280 | 320 | 420
```

Your intuition about seconds was close — the annotations feel like timestamps, but they are **frame indices**, not wall-clock seconds. To convert to seconds: $\text{time} = \text{frame_number} / \text{fps}$ (usually 30fps).

Why Does Annotation Type Multiply Cost?

Annotation Level	What is labeled	Cost per video	Example
Video-level	1 label for whole clip	~\$0.05	"This video contains squats"
Temporal (start/end frames)	N pairs per video	~\$0.50–\$2	"Rep 1: frames 45–135" ← RepCount
Frame-level bounding box	1 box per frame per person	~\$5–20	"Person at [x,y,w,h] in frame 45"
Pixel segmentation	Every pixel labeled per frame	~\$50–200	"These pixels are the person"

RepCount sits at the **second level** — far cheaper than bounding boxes or segmentation, but more expensive than video-level labels because a human annotator had to watch the video and mark exact frame boundaries for every single repetition.

Are the Existing Annotations Done Properly?

Based on research findings: YES — they are research-grade.

- Annotated by ShanghaiTech University for CVPR 2022 (peer-reviewed, Oral presentation)
- Fine-grained, frame-level temporal precision
- Includes anomaly cases: incomplete reps, pauses, inconsistencies — intentionally
- Used by 3+ follow-up papers (PoseRAC, OVR, skeleton-based methods) without re-annotation

How to verify programmatically (audit checklist):

```
python

# 1. Check start < end for all pairs
assert all(row['start_X'] < row['end_X'])

# 2. Check no overlap between consecutive reps
assert all(row['end_X'] < row['start_X+1'])

# 3. Check count matches number of non-null start/end pairs
assert row['count'] == number_of_valid_pairs(row)

# 4. Check all frame indices <= num_frames
assert all(frame_idx <= row['num_frames'])

# 5. Spot-check 20 random videos: watch them and verify manually
```

Do We Need Additional Annotation Tooling?

For the prototype: NO. The existing annotations are sufficient to train and evaluate.

Scenarios where you **WOULD** add tooling:

Scenario	Tool to Use	When
Adding new exercises not in RepCount	CVAT (open source)	Phase 2 scaling
Adding pose-level salient frame labels (like PoseRAC)	CVAT + custom export script	If adopting PoseRAC approach
Correcting annotation errors found in audit	Label Studio (lightweight)	If >5% error rate found in audit

Annotating your own recorded videos

CVAT or VGG Image Annotator

When collecting

(VIA)

proprietary data

◆ PROTOTYPE PLAN — Start Today

Stack

MediaPipe Pose (person detect + 33 keypoints)
+
Joint Angle Feature Extractor (hip-knee-ankle angle time series)
+
Sliding Window LSTM / 1D CNN Counter
+
Peak Detection Post-processing (scipy.signal.find_peaks)

Why This Stack (Not TransRAC)?

- TransRAC requires Video Swin Transformer backbone (heavy, not mobile-friendly)
- MediaPipe + LSTM is: fast to train, runs on mobile, interpretable, < 5MB model size
- PoseRAC (the current SOTA) uses a similar pose-first philosophy — OBO 0.56
- We can always upgrade the counter head later while keeping the pose frontend

Feasibility Go/No-Go Gate Criteria

Criterion	Green	Yellow	Red
MediaPipe detects person	> 95% of squat frames	85–95%	< 85%
Joint angle clearly separates up/down	Visual inspection: yes	Noisy but separable	No signal
Naive peak detection OBO	> 0.40	0.25–0.40	< 0.25
Inference speed (CPU)	< 200ms/frame	200–500ms	> 500ms

■ If ≥ 3 criteria are Green → **Add 3 resources and move to Phase 2 full pipeline.**

◆ **MOBILE CONSIDERATIONS (Baked in from Day 1)**

Decision	Choice	Rationale
Pose framework	MediaPipe (Google)	Runs natively on iOS + Android, TFLite backend
Model format	TFLite / CoreML	Required for on-device inference
Max model size	< 20MB	App store / download experience
Latency target	< 100ms per frame	Enables ≥ 15 FPS real-time counting
Offline support	Yes (on-device only)	No API call needed, works in gym (poor signal)
UI counter display	Overlay on live camera	Real-time feedback loop for user